

Phase 1 — Foundation Completion Report

Monorepo, local infrastructure, API skeleton, database, authentication and CI

STATUS

Complete

COMPLETED

9 October 2026

PREPARED BY

Vivek Shukla

BRANCH

claude/peaceful-fermi-hvmzt4

COMMITTS

f5f2e0e ... 4db65bc (6
commits on top of Phase 0
e851636)

NEXT

Phase 2 — Backend modules

1. Summary

Phase 1 delivers the foundation every later phase builds on: a pnpm + Turborepo monorepo with shared tooling, local infrastructure, a production-shaped NestJS API skeleton, the complete database schema with a realistic seed, full authentication for mobile and portal users, and a CI pipeline.

The whole system runs locally with **zero external accounts** — Firebase and email use mock adapters selected by environment variables.

Verification at sign-off

All checks were run on the final commit against PostgreSQL 16 + pgvector and Redis 7.

Check	Command	Result
Formatting	<code>pnpm format:check</code>	✓ clean
Lint (type-aware, strict)	<code>pnpm lint</code>	✓ 0 errors, 0 warnings
Type-check	<code>pnpm typecheck</code>	✓
Unit tests	<code>pnpm test</code>	✓ 56 passed (API 52, shared-types 4)
Migration drift	<code>pnpm --filter @tirth-now/api db:check</code>	✓ in sync
API end-to-end tests	<code>pnpm test:e2e</code>	✓ 33 passed

Check	Command	Result
Seed (run twice)	<code>pnpm db:seed</code>	✓ idempotent
Production build	<code>pnpm build</code>	✓ API and worker boot from <code>dist/</code>

2. Scope delivered

Checked against the Phase 1 brief.

Requirement	Status	Where
Turborepo + pnpm wiring	✓	<code>package.json</code> , <code>pnpm-workspace.yaml</code> , <code>turbo.json</code>
Shared configs (ESLint, tsconfig, Prettier)	✓	<code>packages/config</code>
Husky + lint-staged	✓	<code>.husky/</code> , <code>.lintstagedrc.cjs</code>
Conventional commits	✓	<code>commitlint.config.cjs</code> , <code>commit-msg</code> hook, <code>Ci job</code>
docker-compose: Postgres + pgvector, Redis, MinIO, Mailpit	✓	<code>infra/docker/docker-compose.yml</code>
Config module with Zod-validated env	✓	<code>apps/api/src/core/config</code>
Prisma module	✓	<code>apps/api/src/core/prisma</code>
Health check	✓	<code>GET /healthz</code> , <code>GET /readyz</code>
Global exception filter	✓	<code>apps/api/src/core/http/all-exceptions.filter.ts</code>
Request ID + pino logging	✓	<code>apps/api/src/core/logging</code>
Rate limiting (throttler + Redis)	✓	<code>apps/api/src/core/throttle</code>
Helmet, CORS	✓	<code>apps/api/src/app.setup.ts</code>
Swagger at <code>/docs</code>	✓	<code>apps/api/src/app.setup.ts</code>
Prisma schema for the full data model	✓	61 tables, 36 enums — <code>apps/api/prisma/schema.prisma</code>
First migration	✓	<code>apps/api/prisma/migrations/20261009065213_init</code>
Seed: 5 temples per city with aarti times	✓	6 cities × 5 temples, 132 aarti schedules, 14 POIs
Seed: sample hotels / cabs / restaurants	✓	2 hotels, 6 room types, 360 room-nights, 3 vehicles, 3 fare classes, 1 restaurant (8 dishes), 1 experience (28 slots)
Seed: 1 admin, 2 vendors, demo user	✓	see §5
Auth: Firebase token exchange	✓	<code>POST /v1/auth/firebase</code>

Requirement	Status	Where
Auth: portal login (+ optional OTP)	✓	POST /v1/auth/portal/login , /portal/otp/verify
Auth: refresh rotation	✓	POST /v1/auth/refresh
Auth: logout	✓	POST /v1/auth/logout
RBAC guards + @Roles()	✓	apps/api/src/core/auth
CI: lint, type-check, test, build on every PR	✓	.github/workflows/ci.yml

Also delivered beyond the brief: password reset for portal users, GET /v1/me , a separate worker endpoint, two custom lint rules that enforce architecture (§3.5), a migration helper with CI drift check, and a dev outbox for mock email.

3. What was built

3.1 Monorepo

```
apps/api           NestJS API (HTTP + worker endpoints)    ← built in Phase 1
packages/config    ESLint / tsconfig / Prettier presets    ← built in Phase 1
packages/shared-types  roles, error codes, money, i18n, auth Zod ← built in Phase 1
apps/mobile, apps/vendor-portal, apps/admin-portal,
packages/api-client, packages/ui          ← placeholders (Phases 3–5)
infra/docker       docker-compose + Postgres init
infra/ci           CI helper scripts
.github/workflows  ci.yml
```

Turborepo runs `build`, `lint`, `typecheck`, `test` and `test:e2e` across packages, building `shared-types` first.

3.2 API skeleton

Concern	Implementation
Configuration	Every env var validated by Zod at boot; the app refuses to start with a clear list of problems. In production it also refuses mock providers (unless listed in <code>ALLOW MOCK PROVIDERS</code>) and the local default JWT secret.
HTTP hardening	Helmet headers, <code>x-powered-by</code> removed, CORS allowlist from <code>CORS_ORIGINS</code> , <code>trust proxy</code> configurable.
Routing	All routes under <code>/v1</code> ; <code>/healthz</code> and <code>/readyz</code> unprefixed.
Validation	Global pipe: whitelist + reject unknown fields + transform. Failures return field-level details.
Errors	One envelope everywhere: <code>{ "error": { "code", "message", "details?", "requestId" } }</code> . Prisma unique/not-found errors map to 409/404; unknown errors become <code>INTERNAL</code> without leaking internals.
Logging	pino JSON (pretty in development). Request id reused from a well-formed <code>x-request-id</code> or generated, echoed in the response. Tokens, passwords, codes, emails and phone numbers are redacted.

Concern	Implementation
Rate limiting	Redis-backed. Default bucket for all routes, plus a stricter auth bucket (<code>THROTTLE_AUTH_PER_MIN</code> , default 10/min) on auth endpoints. Returns <code>RATE_LIMITED</code> .
Health	<code>/healthz</code> = liveness; <code>/readyz</code> checks Postgres and Redis and returns 503 with per-dependency status when either is down.
Docs	Swagger UI at <code>/docs</code> , JSON at <code>/docs-json</code> ; disabled in production unless <code>SWAGGER_ENABLED=true</code> .
Worker	<code>dist/worker.js</code> — same image, health endpoints only for now; BullMQ processors arrive in Phase 2 (ADR-0005).
Dev tools	<code>GET /v1/dev/outbox</code> shows mock-sent email; not registered in production.

3.3 Provider layer

Every external service sits behind an interface with a real and a mock adapter (ADR-0003). Phase 1 implements the two that auth needs:

Provider	Selector	Real adapter	Mock behaviour
<code>FirebaseAuthProvider</code>	<code>FIREBASE_AUTH_PROVIDER=mock firebase</code>	firebase-admin, checks token revocation	Accepts <code>mock:<uid>[:<+E164>][:<email>]</code> and <code>mock:guest:<uid></code>
<code>EmailProvider</code>	<code>EMAIL_PROVIDER=mock smtp</code>	nodemailer SMTP (Mailpit locally)	Keeps the last 200 messages in memory

Real adapters run through `callWithPolicy()` : per-call timeout, exponential backoff with jitter, retries only on network errors, 429 and 5xx.

3.4 Authentication

Flow	Behaviour
Mobile sign-in	App sends a Firebase ID token once. The API verifies it, creates or finds the user, registers the device (FCM token moves to whoever signed in last on a shared phone) and returns an access + refresh token pair.
Account linking	An existing account with the same <i>verified</i> phone or email is linked instead of duplicated.
Guests	Anonymous sign-in creates a guest; signing in later with the same Firebase uid and a phone upgrades the same account in place. <code>@NoGuests()</code> routes reject guests.
Portal sign-in	Email + argon2id password. Unknown emails take the same time and return the same error as wrong passwords.
OTP (optional)	Users with <code>mfa_enabled</code> get a 6-digit emailed code: single use, 5-minute expiry, locked after 5 wrong attempts.
Access tokens	JWT, 15 minutes, carries <code>roles</code> , <code>vendorIds</code> , session id and guest flag.

Flow	Behaviour
Refresh tokens	256-bit opaque, 30 days, stored only as SHA-256, rotated on every use . Presenting an already-used token revokes the whole session (theft detection). Concurrent refreshes: exactly one wins. Suspended users cannot refresh.
Logout	Revokes the current session, or every session with <code>allSessions: true</code> .
Password reset	Always answers 202 (no email enumeration); 30-minute single-use link to the vendor or admin portal; resetting signs out every session.
Authorization	Global guards in order: throttle → authenticate → authorize. Every route requires a token unless <code>@Public().@Roles()</code> with hierarchy: <code>super_admin</code> $\supseteq$ <code>admin</code> , <code>vendor_owner</code> $\supseteq$ <code>vendor_staff</code> .

3.5 Architecture enforcement

Rule	Enforced by
Modules import each other only through <code>index.ts</code>	ESLint rule <code>tn/module-boundaries</code>
<code>core/</code> never imports <code>modules/</code>	same rule
Vendor SDKs (firebase-admin, nodemailer, razorpay, openai, ...) only inside <code>core/providers</code>	ESLint <code>no-restricted-imports</code>
No <code>any</code> , strict type-aware rules	<code>typescript-eslint</code> <code>strictTypeChecked</code>
Schema changes always ship with a migration	<code>db:check</code> in CI

3.6 Database

- **61 tables, 36 enums** covering identity, catalog, vendors, inventory, orders, payments, reviews & reels, AI and ops — UUID keys, UTC `timestampz`, integer paise.
- Extensions: `pgcrypto`, `pg_trgm`, `unaccent`, `citext`, `vector`.
- **Integrity in the database itself** (added in raw SQL, ADR-0008):
 - CHECK constraints: room inventory can never go below zero or above total; experience bookings never exceed capacity; all money non-negative; payments and refunds positive; commission 0–100%; ratings 1–5.
 - A deferred trigger rejects any ledger transaction that does not balance.
 - Ledger, audit log and order status history are append-only (UPDATE/DELETE blocked).
 - HNSW vector index for AI retrieval, trigram indexes for fuzzy temple search, partial index for expiring booking holds.
- The five RBAC roles are inserted by the migration, so they always exist.

3.7 Seed data

Idempotent (deterministic IDs) and refuses to run against production.

Data	Contents
Cities	Mathura, Vrindavan, Goverdhan, Barsana, Gokul, Nandgaon (EN + HI)
Temples	5 per city, bilingual names, coordinates, tags, crowd level

Data	Contents
Aarti schedules	Per temple; standard and Pushtimarg patterns; Banke Bihari has separate summer/winter timings
POIs	Hospitals, police stations, parking, pharmacy, ghats; emergency flags
Parikrama	Goverdhan Parikrama (21 km) with 4 stops
Hotel vendor	<i>Shri Radha Residency</i> — 2 hotels, 3 room types each, 60 nights of inventory (weekend +20%)
Cab + food vendor	<i>Braj Yatra Cabs & Kitchen</i> — 3 vehicles, 3 fare classes with fixed Mathura→Vrindavan fares, a satvik restaurant with 8 dishes, a Yamuna boat ride with 28 slots
Platform	Commission rules (10% default, 15% food, 8% cab), notification templates, feature flags, national helplines (112, 108, 1091, 1363)

4. Testing

Suite	Count	Covers
Config	13	defaults, parsing, all errors reported, production guards, Firebase key handling
Roles guard	11	public routes, role hierarchy, guest blocking
Error filter	5	envelope mapping incl. throttling and Prisma errors, no internal leakage
Call policy	10	retries, no retry on 4xx, retry budget, timeouts
Mock Firebase	11	every token format, malformed tokens
Migration helper	2	strips only <code>rx_</code> drops
shared-types money	4	paise conversion, parsing, en-IN formatting, banker's rounding
E2E — health & HTTP	8	health/ready, OpenAPI, request ids, error envelope, 404s, security headers, CORS
E2E — auth	25	sign-in, linking, guest upgrade, device/FCM moves, validation, rotation, reuse revocation, concurrent refresh, expiry, suspension, logout, portal login, RBAC, OTP single-use and lockout, password reset, tampered tokens, rate limiting

E2E tests run against a real Postgres and Redis (DB 15), migrated once and reset between tests.

5. Running it locally

```
cp .env.example .env
pnpm install
pnpm setup      # docker compose up + migrations + seed
pnpm dev        # API on http://localhost:4000 – Swagger at /docs
```

Who	Credentials (local only)
Admin (super_admin)	<code>admin@tirthnow.local</code> / <code>TirthNow@Admin123</code>
Hotel vendor	<code>hotel.owner@tirthnow.local</code> / <code>TirthNow@Vendor123</code>
Cab + food vendor	<code>cabs.owner@tirthnow.local</code> / <code>TirthNow@Vendor123</code>
Mobile demo user	Firestore token <code>mock:demo-user:+919999900001</code>

Local services: MinIO console <http://localhost:9001>, Mailpit <http://localhost:8025>.

6. Decisions recorded

ADR	Decision
0007	Pin the newest releases of mature majors (NestJS 11, Prisma 6, TypeScript 5.9, ESLint 9, Jest 29, Zod 3) instead of brand-new majors; upgrade one at a time later
0008	Database objects Prisma cannot model live in migrations as raw SQL (<code>rx_</code> prefix); new migrations go through <code>pnpm db:migrate:new</code> ; CI checks drift

Earlier ADRs 0001–0006 (modular monolith, Firebase + own JWT, provider interfaces, order state machine, worker process, i18n/geo) remain in force.

7. Known limitations and deferred items

Item	Status / plan
OTP by SMS	Email only in Phase 1; SMS arrives with the notifications module (Phase 2)
Other providers (payments, SMS, maps, AI, storage, ...)	Implemented with their modules in Phase 2
Idempotency-Key interceptor	Phase 2 (first needed by orders, payments and SOS)
Circuit breaker in the call policy	Phase 2, with the first high-volume adapters
Soft-delete query filtering	Phase 2, with the first module that soft-deletes
Production Dockerfiles, deploy workflow	Phase 6
Sentry wiring	Phase 6 (hook point exists in the exception filter)
Access-token revocation	By design, a revoked session's access token stays valid for up to 15 minutes (ADR-0002)
Concurrent refresh	Clients must serialise refresh calls; the second of two simultaneous refreshes is treated as reuse
Docker in the build sandbox	Compose file validated with <code>docker compose config</code> ; the same Postgres 16 + pgvector and Redis were run natively for all tests

Item	Status / plan
CI on GitHub	The workflow runs on pull requests and pushes to <code>main</code> ; it has not run yet because no PR exists

8. Actions needed from the product owner

From `_CLIENT_SETUP_TODO.md` , the items that matter now:

1. **Verify seeded aarti times and temple data** (`apps/api/prisma/seed/catalog-data.ts`). They are realistic samples, not confirmed timings, and the AI guide will quote them.
2. **Decide the Node.js version.** The spec fixes Node 20, which reached end-of-life in April 2026.
Recommendation: Node 22 LTS.
3. **Start long-lead accounts now:** Razorpay with Route, MSG91 + TRAI DLT templates, WhatsApp Business verification, Apple Developer enrolment.

9. Next: Phase 2

Backend modules, each with controller, service, DTOs and tests: identity (profile, devices, emergency contacts), catalog with search, files, vendors and KYC, inventory with date-level locking, orders state machine, payments (Razorpay mock, webhooks, refunds, commission, ledger, payouts), realtime, AI, reels, safety/SOS, notifications, weather, transit and admin.